OS CPU Scheduling Tutor Chatbot

Retrieval-Augmented Generation (RAG) + Instruction-Tuned LLM

Operating Systems Lab: Final Project Report

Name	Registration No.	Course	Date
R G Rishon	24BCE2061	Operating Systems Lab	12 October 2026

About the project

This project is a tutor chatbot for the **CPU Scheduling** unit (FCFS, SJF, SRTF, Priority and Round Robin). General chatbots often **guess** scheduling answers and can give two different answers to the same question. This tutor avoids that by working in three parts:

- **A simulator** calculates every number (Gantt chart, waiting time, turnaround time), so numbers are never guessed.
- **A search step** finds the right passage in the course notes, so explanations come from the course material.
- **A language model** (Qwen2.5-3B, trained further on scheduling questions) writes the answer in simple words. A **safety check** removes any answer that contains a wrong formula or made-up numbers.

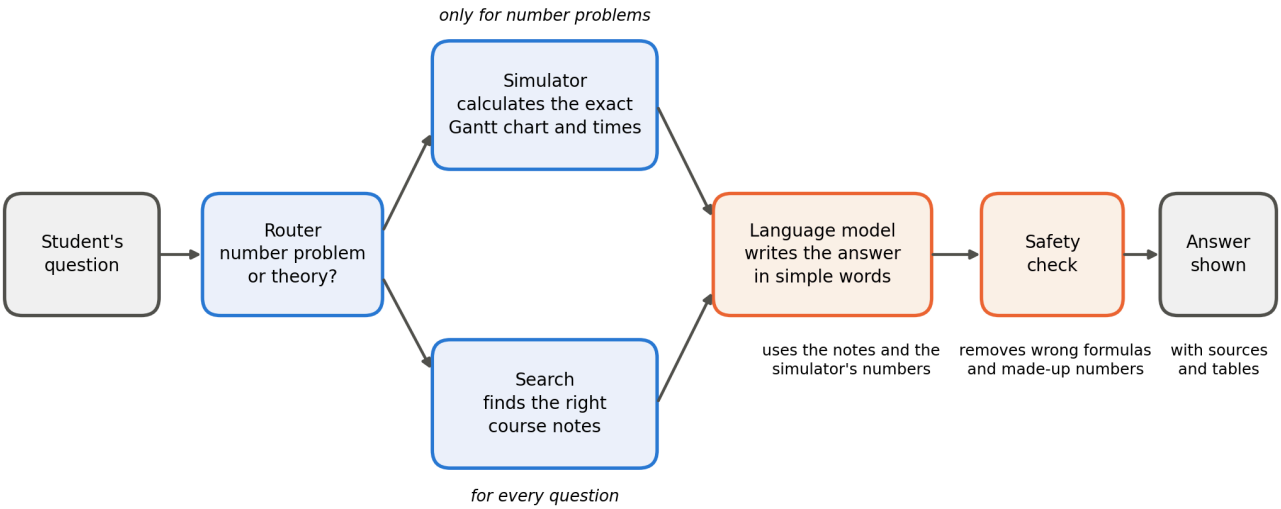


Figure 1: How a question is answered.

1. Dataset

The project uses three sets of data: one for the chatbot to search, one to train the model, and one kept aside for testing.

Dataset	What it is for	Size	Where it came from
Knowledge base	Text the chatbot searches to find answers	451 passages (about 59,000 words) from 24 files	My course notes (244 passages) OSTEP textbook chapters (95) Wikipedia articles (48) 9 GeeksforGeeks pages (64)
Training set	Question and answer examples used to train the model	1,027 examples	Theory answers written by hand; numerical answers made and checked by the simulator
Test sets	Questions used only to measure how well the system works	55 questions + 109 questions	Written after the training data was fixed; none of them is in the training set

What is inside the 1,027 training examples

Type of example	Count	Example question
Theory: concepts, comparisons, common mistakes	411	"Explain Round Robin with an analogy." "A student says Round Robin causes starvation. Is this correct?"
Numerical: full solution	240	"Solve this SJF problem and draw the Gantt chart."
Numerical: one specific answer	260	"What is the waiting time of P2?" "How many context switches occur?"
Practice problems	40	"Generate a Priority scheduling problem with 5 processes."
Check my answer	48	"Check my answer: average waiting time = 16."
Off-topic questions (the bot refuses)	28	"What is the capital of France?"

Licences. OSTEP is free from its authors and Wikipedia is open (CC BY-SA). The GeeksforGeeks pages are copyrighted, so they are used only on my own computer, as short quotes with a link, and are not part of the submitted files. Faculty slides and past papers were planned in the progress report but were not available in digital form, so I wrote my own notes using the textbook's conventions (for example, a lower priority number means higher priority).

2. Dataset Preprocessing

Raw text and raw examples cannot be used directly. They were cleaned and checked in the steps below.

Knowledge base

1. **Clean:** remove page numbers and running headers, join words split across lines, fix special characters.
2. **Cut into passages:** about 300 words each, with 45 words overlapping so a worked example is not cut in half. Each stored question-and-answer is kept whole.
3. **Label and index:** each passage is tagged with its topic (FCFS, SJF, ...) and indexed for keyword search.

4. **Web pages:** downloaded politely (obeying robots.txt and waiting between requests); only the article text is kept, the website menu is dropped, and one section that used the opposite priority rule was removed.

clean_chunk.py · clean_text: tidying raw text before it is used

```python
def clean_text(raw: str) -> str:
    """Remove slide/page artefacts, fix hyphenation, normalise whitespace."""
    text = unicodedata.normalize("NFKC", raw)          # "fi" ligature -> "fi"
    text = text.replace("\r\n", "\n").replace(" ", " ")
    # join words broken across a line by a hyphen: sched-\nuling -> scheduling
    text = re.sub(r"(\w+)-\n(\w+)", r"\1\2", text)
    lines = []
    for line in text.split("\n"):
        if any(re.match(p, line, flags=re.IGNORECASE) for p in HEADER_NOISE):
            continue
        lines.append(line.strip())
    text = "\n".join(lines)
    text = re.sub(r"\s*\(Source:[^)]*\)", "", text)  # citation tags are re-added from metadata
    text = re.sub(r"^=+\s*(.*?)\s*=+$", r"\n\1.", text, flags=re.MULTILINE)  # wiki "== Heading =="
    text = re.sub(r"^#+\s*(.*?)([.?!:]?)$", lambda m: m.group(1) + (m.group(2) or "."),
                  text, flags=re.MULTILINE)                                  # markdown "## Heading"
    text = re.sub(r"\n{3,}", "\n\n", text)             # collapse blank runs
    text = re.sub(r"[ \t]{2,}", " ", text)
    return text.strip()
```

Figure 2: Code screenshot: cleaning the raw text (clean_chunk.py).

clean_chunk.py · chunk_text: paragraph-aware chunks, 300 words with 45-word overlap

```python
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    chunks, current = [], []
    for para in paragraphs:
        words = para.split()
        if len(current) + len(words) <= chunk_words:
            current.extend(words)
            continue
        if current:
            chunks.append(" ".join(current))
        if len(words) > chunk_words:                   # paragraph alone is too long
            step = chunk_words - overlap_words
            for i in range(0, len(words), step):
                piece = words[i:i + chunk_words]
                if piece:
                    chunks.append(" ".join(piece))
                if i + chunk_words >= len(words):
                    break
            current = []
        else:
            tail = current[-overlap_words:] if current else []
            current = tail + words
```

Figure 3: Code screenshot: cutting the notes into overlapping passages (clean_chunk.py).

rag/fetch_wiki.py · allowed(): the web downloader checks robots.txt first

```python
def allowed(url: str) -> bool:
    """True if robots.txt lets a generic client fetch this URL (unknown robots.txt means allowed)."""
    parts = urllib.parse.urlparse(url)
    base = f"{parts.scheme}://{parts.netloc}"
    if base not in _robots:
        rp = urllib.robotparser.RobotFileParser()
        try:
            _wait()
            r = requests.get(base + "/robots.txt", headers={"User-Agent": USER_AGENT}, timeout=20)
            if r.status_code == 200:
                rp.parse(r.text.splitlines())
                _robots[base] = rp
            else:
                _robots[base] = None              # no robots.txt: nothing is disallowed
        except requests.RequestException:
            _robots[base] = None
    rp = _robots[base]
    return True if rp is None else rp.can_fetch(USER_AGENT, url)
```

Figure 4: Code screenshot: the web downloader checks robots.txt before it fetches a page (rag/fetch_wiki.py).

Training set

#	Step	Result
1	**Check the format:** every example has a question and an answer	1,027 read, 0 broken
2	**Remove duplicates:** the same question written twice	0 duplicates left
3	**Remove poor answers:** too short or cut off	0 removed from the final set
4	**Verify the numbers:** the simulator recalculates every worked solution and compares it with the answer	1,027 correct, 0 failed
5	**Split:** 90% for training, 10% for checking during training. Different wordings of the same question stay on the same side	924 training, 103 validation
6	**Match the real prompt:** each example is written the way the app asks the model (retrieved notes + simulator output)	In the training file: 820 with notes attached, 104 plain

The checks were tested on a deliberately faulty sample file. A duplicate, a cut-off answer and a wrong worked solution (12.0 written, 17.0 correct) were all caught.

prepare_dataset.py · every worked solution is recomputed by the simulator before it is kept

```python
    procs = [Process(*p) for p in meta["processes"]]
    algo = meta["algorithm"].lower()
    if algo == "fcfs":
        result = fcfs(procs)
    elif algo == "sjf":
        result = sjf(procs, preemptive=meta.get("preemptive", False))
    elif algo == "srtf":
        result = sjf(procs, preemptive=True)
    elif algo == "priority":
        result = priority_scheduling(procs, preemptive=meta.get("preemptive", False))
    elif algo == "rr":
        result = round_robin(procs, meta["quantum"])
    else:
        return False, f"unknown algorithm {algo!r}"
```

Figure 5: Code screenshot: every worked solution is recalculated by the simulator (prepare_dataset.py).

Step 6 matters because the model should learn the task it will really be given. Each example is written with the same notes and simulator output that the app supplies when it asks the model a question.

rag_format.py · each training example is written in the same prompt the app uses

```python
    def _theory(self, rec, text, meta):
        src = self.by_title.get(meta.get("source_title"))
        if src is None:
            return None
        hits = self.retriever.search(text, algorithm=route(text).topic)
        pos = next((i for i, h in enumerate(hits) if h.get("title") == src["title"]), None)
        if pos is None:                              # make sure the answer's own passage is in the context
            slot = int(hashlib.md5(text.encode()).hexdigest(), 16) % len(hits)
            hits = hits[:slot] + [dict(src, score=1.0)] + hits[slot:len(hits) - 1]
            pos = slot
        answer = SOURCE_TAG.sub("", rec["output"]).strip() + f" [{pos + 1}]"
```

Figure 6: Code screenshot: a training example is built from the real retrieved notes, and its answer ends with the source number (rag_format.py).

3. Code Implementation

All code is in the project folder **ostutor/**: 28 Python files (about 5,500 lines) and 38 automatic tests. Running `python run_demo_checks.py` shows every test passing. The code screenshots below are taken from these files.

File	What it does
scheduler.py	The simulator: FCFS, SJF, SRTF, Priority (both kinds) and Round Robin, with Gantt chart, times and a step-by-step log
router.py, intent.py	Decide whether a question is a number problem or a theory question; read process tables typed by the student
build_index.py, retriever.py	Build and search the knowledge base; refuse questions outside CPU scheduling
tutor.py	Joins everything: simulator + search + language model + safety check
checker.py, practice.py	Mark a student's answer and show the first wrong step; create practice problems
app.py, present.py	The web page (Streamlit): shows answers as a short summary, steps, formulas and tables
build_training_data.py	Creates, checks and splits the 1,027 training examples
finetune/ (Colab notebook)	Trains the language model

Reading the question

Before answering, the router decides what the student is asking: a practice problem, a number problem (it reads the processes typed in the question and runs the simulator), a check of the student's own answer, or a theory question.

router.py · route(): deciding what kind of question the student asked

```python
def route(text):
    low = text.lower()
    algo = detect_algorithm(text)
    if re.search(PRACTICE, low):
        return Route("practice", algorithm=algo, quantum=detect_quantum(text))
    try:
        procs, notes = parse_processes(text)
    except ValueError as e:                # e.g. a burst time of 0
        return Route("concept", algorithm=algo, notes=[f"could not use the process data: {e}"])
    if re.search(CHECK, low) and procs:
        r = Route("check", algorithm=algo, quantum=detect_quantum(text), processes=procs, notes=notes)
        return r
```

Figure 7: Code screenshot: deciding what kind of question was asked (router.py).

Finding the notes

The search looks in the vetted notes first. Web pages are searched separately and can only add an extra card, so they can never replace a vetted answer.

retriever.py · search(): your notes and scraped web text are searched separately

```python
        hits = []
        q_tokens = canon_tokens(query)
        query = expand_query(query)
        for i, s in self._scores(query, n=len(self.records) if origin == "web" else 50):
            r = dict(self.records[i])
            is_web = r["source"].startswith("web/")
            if (origin == "vetted" and is_web) or (origin == "web" and not is_web):
                continue
```

Figure 8: Code screenshot: vetted notes and web pages are searched separately (retriever.py).

Simulator

The simulator follows the textbook rules. In Round Robin, a process that arrives during a time slice joins the queue **before** the process that was just stopped goes back to the end. This is the most common place for mistakes in exams.

scheduler.py · Round Robin: arrivals join the queue before the preempted process

```python
        arrived = []
        while i < n and order[i].arrival <= time:  # arrivals during the slice
            queue.append(order[i]); arrived.append(order[i]); i += 1
        if arrived:
            log.append(f"t={time}: {_names(arrived)} arrived during the slice and join the queue first")
        if cur.remaining > 0:
            queue.append(cur)
            log.append(f"t={time}: quantum expires, {cur.pid} goes to the back "
                       f"(remaining {cur.remaining})")
        else:
            cur.completion = time
            done += 1
            log.append(f"t={time}: {cur.pid} completes")
```

Figure 9: Code screenshot: the Round Robin queue rule (scheduler.py).

Training the language model

The model is **Qwen2.5-3B-Instruct**. It was trained further with **QLoRA**, a method that trains only a small add-on layer, so it fits on the free Google Colab GPU. It learned from the 924 examples for 2 rounds

(epochs), and it learned from the answers only. The result is a 1.93 GB file that runs on my laptop with Ollama, so it works offline.

ostutor_qlora_colab.ipynb · QLoRA training settings (cell 6)

```
use_bf16 = torch.cuda.is_bf16_supported()
cfg = dict(
    dataset_text_field="text",
    per_device_train_batch_size=2, gradient_accumulation_steps=4,
    num_train_epochs=2, learning_rate=1e-4, warmup_ratio=0.05, lr_scheduler_type="cosine",
    weight_decay=0.01, logging_steps=10,
    save_strategy="epoch", save_total_limit=1,
    load_best_model_at_end=True, metric_for_best_model="eval_loss", greater_is_better=False,
    fp16=not use_bf16, bf16=use_bf16, optim="adamw_8bit", seed=42,
    output_dir="outputs", report_to="none")
```

Figure 10: Code screenshot: training settings in the Colab notebook.

Safety check

A small language model sometimes invents things. After the model answers, the code checks the answer. If it contains a wrong formula, or a number that is not in the course notes or the question, the answer is replaced by the vetted course answer.

tutor.py · safety guard: numbers that are not in the course text are treated as invented

```
_NUM = re.compile(r"(?<![\w.])\d+(?:\.\d+)?(?![\w])")


def ungrounded_numbers(text, context, question):
    """Numbers in a concept answer that appear neither in the question nor in the retrieved course text:
    a sign the model invented an example. (Citation markers and list numbering are ignored.)"""
    body = re.sub(r"\[\d+\]", " ", text)
    body = re.sub(r"(?m)^\s*\d+[.)]\s", " ", body)
    known = set(_NUM.findall(context)) | set(_NUM.findall(question)) | {"0", "1"}
    return sorted(set(_NUM.findall(body)) - known)
```

Figure 11: Code screenshot: finding numbers that are not in the course text (tutor.py).

Checking a student's own answer

The student's Gantt chart is compared with the simulator's, one time unit at a time. The checker stops at the first difference, because every later mistake follows from it, and then explains what the scheduler should have done at that moment.

checker.py · check_answer(): stops at the first step where the student's chart differs

```
    # 1. Gantt chart: the earliest divergence explains everything after it
    if ans["gantt"]:
        mine, truth = _timeline(ans["gantt"]), _timeline(result["gantt"])
        end = max(max(mine, default=0), max(truth, default=0)) + 1
        for t in range(end):
            if mine.get(t) != truth.get(t):
                expected, got = truth.get(t, "nothing (all done)"), mine.get(t, "nothing")
                first_error = (f"Your Gantt chart first diverges at t={t}: you have {got} on the CPU, "
                               f"but {expected} should run.")
                findings.append(first_error)
                ctx = _context(result["log"], t)
                if ctx:
                    findings.append("What the scheduler sees at that point:\n" +
                                    "".join(f"\n    - {c}" for c in ctx))
                break
        else:
            findings.append("Gantt chart: correct.")
```

Figure 12: Code screenshot: finding the first step where the student's chart is wrong (checker.py).

4. Metrics (Assessment): Results and Discussion

Tests of the whole system

Test	Result
Simulator against the 5 textbook examples (average waiting time 17.00, 7.00, 6.50, 5.67, 8.20)	**5 of 5** exact
30 new numerical questions, answers checked against the simulator	**30 of 30** correct
Routing: number problem or theory, and which algorithm (55 questions)	**55 of 55** correct
Finding the right notes for 20 theory questions	**20 of 20** found
Refusing off-topic questions (5 questions)	**5 of 5** refused
109 mixed student-style questions (full system, trained model)	**108 of 109** correct
Automatic tests of the code	**38 of 38** pass



Figure 13: The simulator's output for the textbook example (P1 = 24, P2 = 3, P3 = 3, quantum 4). Average waiting time 5.67, as in the textbook.

Trained model compared with the untrained model

Both models answered the same 109 questions. Stored answers were switched off, so **every answer was written by the model**.

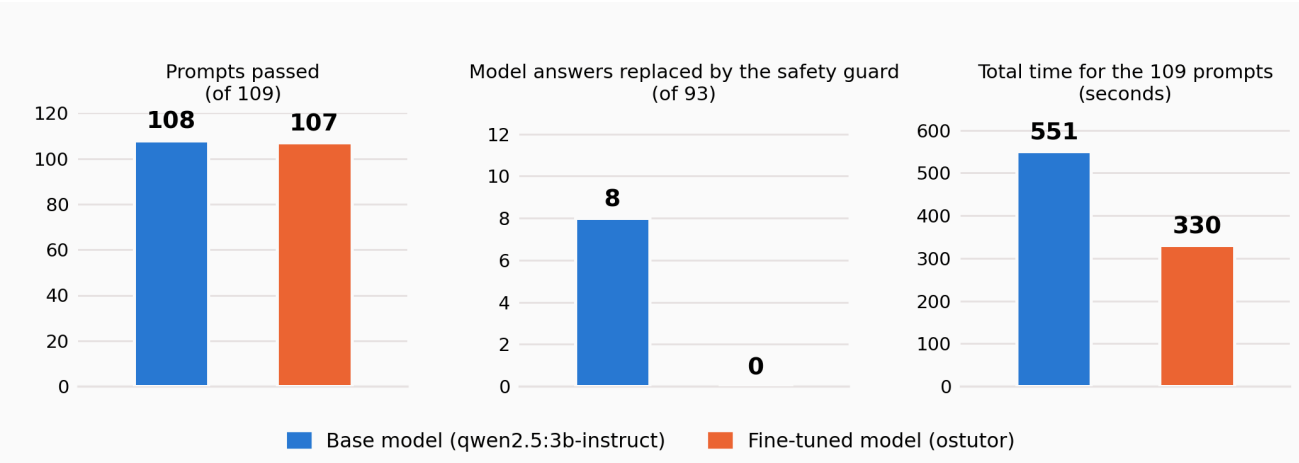


Figure 14: Untrained (base) model against the trained model on the same 109 questions.

Discussion: what the results show

1. **The numbers are right because the simulator calculates them,** not because the model learned them. All 30 new numerical questions were correct.
2. **Training did not raise the score.** The trained model passed 107 to 108 of 109 (three runs) and the untrained model passed 108 (one run). On accuracy alone they are equal.

3. **Training made the model safer and faster.** The safety check had to replace 8 of 93 answers from the untrained model (wrong formula or made-up numbers) and none from the trained model. It also took 330 seconds instead of 551.
4. **Most mistakes came from gaps in my notes, not from the model.** Both models said "SRTF is not optimal" until I added the missing note. Improving the notes helped more than training.
5. **Limits of these results.** I wrote the test questions myself, so the scores may be better than with outside questions. The training graph from Colab was not saved. Results change by about one question between runs.

5. What Went Wrong

What went wrong	Why	What I did
First training attempt failed. The text length showed as "1", training loss (0.10) was far below validation loss (0.81), then the code crashed	Gemma 3 reads text as images; only 236 examples; a faulty line of code	Switched to Qwen2.5-3B, used 924 examples, removed the faulty line, added a check that stops training if the text is not read properly
Wrong or made-up answers. For example "waiting time = completion – burst – 1" with an invented example	The search had not found the formula note, and a small model fills the gap	Added formula notes, a short list of definitions in every prompt, and the safety check
Search found the wrong note, or showed a whole section instead of an answer	"RR" did not match "Round Robin"; section headings were treated as answers	Matched abbreviations, improved title matching, and allowed only question-style notes as answers
Display problems: stray HTML tags, a confusing sentence, old copies of the app still running	A formatting bug; careless wording; several copies started by mistake	Removed HTML tags, rewrote the sentences, ran one copy only
Web pages: the website menu got mixed in, and one section used the opposite priority rule	Page layout; different books use different conventions	Kept only the article text, dropped the conflicting section, used web text only as an extra
Colab problems: GPU limit reached; the 2 GB download kept failing; the training graph was not saved	Free-tier limits	Recovered the files from the session that was still running. Lesson: save to Google Drive early and save the training log
Plan changed: Qwen3-4B and meaning-based search were planned; faculty slides were not available	Qwen2.5-3B runs better on my laptop; the search libraries were not installed	Used Qwen2.5-3B and keyword search; the code for meaning-based search is written but not switched on

6. What Could Be Done Better (Alternative Flow and Improvements)

- **Better search:** use meaning-based search (bge + FAISS, already coded) together with the keyword search.
- **Better testing:** use past exam questions and classmates' questions, mark the answers by hand, repeat the runs, and save the training graphs.
- **Better model:** retrain on the final notes; try a larger model (Qwen3-4B or 8B) when a GPU is available; teach it to prefer the correct, safe answer.
- **Smarter question reading:** replace the rule-based reader of process tables with a small classifier, or let the model call the simulator as a tool.

- **Stronger safety check:** check every sentence against the notes, not only formulas and numbers.
- **More content:** add faculty slides, the lab manual and past papers; extend to deadlocks, memory management and disk scheduling with the same steps; add a progress tracker for each student.

Conclusion

The tutor solves scheduling problems exactly, explains concepts from the course notes with sources, and refuses questions outside the unit. The trained model runs offline on a laptop. The main remaining work is testing with questions written by other people.

Files: ostutor/ (code), finetune/ostutor_qlora_colab.ipynb (training notebook), finetune/TRAINING_RECORD.md (training record), data/ (datasets), results/ (test results).